Assignment 1A: Exploration

Sertan Erdogan (s5152283) & Hossein Kelisa (sXXXXXX)
Group Number: 84

1 Introduction

Reinforcement Learning (RL) is a type of Machine Learning where an agent learns to interact with the environment to maximize rewards based on its actions [1]. Unlike supervised learning, the agent does not receive a labeled dataset of correct actions; instead, it must discover the optimal strategy through trial and error.

The main challenge in this process is to find the balance between *exploration* and *exploitation*. Exploitation involves selecting the action that currently appears to yield the highest reward based on prior experience. Exploration involves selecting unknown actions to gather more information about the environment, which can lead to higher rewards by discovering better actions.

A simplified yet fundamental version of this challenge is the multi-armed bandit problem. Unlike the full RL setting, a bandit environment involves a "one-step" decision process where there are no state transitions, meaning each action is independent and leads to an immediate reward. This is often compared to a gambler choosing between multiple slot machines (arms), each with an unknown payout distribution.

In this report, we implement and evaluate three distinct policies designed to address this tradeoff: ϵ-greedy, Optimistic Initialization, and Upper Confidence Bound (UCB). We test these algorithms in a bandit environment modeled as a movie recommender system to compare their learning efficiency and performance stability. The remainder of this report presents the methodology, experimental results, discussion, and conclusions.

2 Methodology

2.1 Bandit Problem

The multi-armed bandit is a simplified reinforcement learning framework where an agent chooses between k different actions to maximize cumulative reward. In this study, the environment is a movie recommendation system with $k = 25$ arms. Each arm represents a movie with an underlying average rating $\mu_a \in [1, 5]$ and a standard deviation $\sigma_a \in [0, 3]$. When an action is taken, the environment returns a noisy reward R, which is clipped and rounded to the nearest integer star rating. The agent estimates these hidden means through interaction over $T = 1000$ time steps.

2.2 Exploration Problem

A core challenge in reinforcement learning is the exploration-exploitation tradeoff. **Exploitation** refers to selecting the action with the highest current estimated value, $A_t = \arg\max_a Q_t(a)$, to maximize immediate reward. **Exploration** involves selecting less-known actions to gather information, improving future estimates and potentially increasing long-term reward. Striking a balance between these two behaviors is critical: too much exploitation may cause the agent to miss better options, while excessive exploration may reduce short-term performance.

2.3 ϵ-greedy

The ϵ-greedy agent is implemented in the `eGreedyAgent` class. Action selection is performed as follows:

- With probability ϵ, select a random action (exploration) using `np.random.randint(self.n_actions)`.
- With probability $1-\epsilon$, select the greedy action: `np.argmax(self.Q)`.

The Q-values are updated using the sample-average formula:

$$Q[a] \leftarrow Q[a] + \frac{1}{n[a]}(R_t - Q[a]),$$

where $n[a]$ is the number of times action a has been selected. The ϵ values tested are $[0.01, 0.05, 0.1, 0.25]$. This approach is simple to implement but explores randomly, which may be inefficient in large action spaces.

2.4 Optimistic Initialization

The OI agent, implemented in `OptimisticInitializationAgent`, initializes all action values to a high value:

$$Q[a] = \text{initial_value}.$$

Actions are selected greedily:

$$A_t = \arg\max_a Q[a].$$

As actual rewards are typically lower than the initial value, the agent is naturally driven to explore each arm. After selecting an action, the Q-value is updated using the same sample-average update:

$$Q[a] \leftarrow Q[a] + \frac{1}{n_{\text{obs}}[a]}(R_t - Q[a]),$$

where $n_{\text{obs}}[a]$ counts how many times arm a has been chosen. Initial values tested were $[0, 1, 2, 5, 10]$. OI encourages structured exploration without randomness, but is sensitive to the choice of initial value.

2.5 Upper Confidence Bound (UCB)

The Upper Confidence Bound algorithm balances exploration and exploitation using a confidence bonus term [2]. The UCB agent is implemented in the `UCBAgent` class. It selects actions based on both the estimated value and uncertainty:

$$A_t = \arg\max_a \left[Q[a] + c\sqrt{\frac{\ln t}{n[a]}}\right],$$

where $n[a]$ counts the number of times action a has been selected and c controls exploration intensity. Actions that have not been tried yet are selected first to ensure initial exploration. Q-values are updated incrementally after observing reward R_t:

$$Q[a] \leftarrow Q[a] + \frac{1}{n[a]}(R_t - Q[a]).$$

We explored c values $[0.01, 0.5, 1.0, 2.0, 5.0, 10.0]$. UCB provides a principled exploration mechanism but requires careful tuning of c.

2.6 Hyperparameter Ranges

We evaluated the following hyperparameters:

- ϵ-greedy: $\epsilon \in \{0.01, 0.05, 0.1, 0.25\}$
- Optimistic Initialization: $Q_0 \in \{0, 1, 2, 5, 10\}$
- UCB: $c \in \{0.01, 0.5, 1.0, 2.0, 5.0, 10.0\}$

The detailed results of these parameter studies, including learning curves and performance comparisons, are presented in the Results section.

2.7 Value Update Rule

All agents maintain action-value estimates $Q[a]$ that are updated incrementally using observed rewards R_t. Specifically, after selecting action a, the update is:

$$Q[a] \leftarrow Q[a] + \alpha\big(R_t - Q[a]\big),$$

where $\alpha = 1/n[a]$ for the sample-average approach. Here, $n[a]$ is the number of times action a has been chosen. This ensures $Q[a]$ represents the arithmetic mean of all observed rewards for that action, without storing the full history. In the code:

- For `eGreedyAgent`, `self.n[action]` counts selections.
- For `OptimisticInitializationAgent`, `self.n_obs[action]` counts selections.
- For `UCBAgent`, `self.n[action]` counts selections.

3 Results

3.1 ϵ-greedy

The performance of the ϵ-greedy agent strongly depends on the choice of ϵ. Small values of ϵ (e.g., 0.01) favor exploitation, allowing the agent to quickly accumulate rewards from the best-known arm. However, insufficient exploration may cause the agent to overlook better arms, particularly if the initial estimates are inaccurate. Larger values of ϵ (e.g., 0.25) increase the frequency of random exploration, which helps discover high-reward arms early but can reduce cumulative reward in the short term due to frequent suboptimal selections.

Across the tested ϵ values, we observed that moderate values (0.05–0.1) achieved the best balance between exploration and exploitation, resulting in higher cumulative rewards over the $T = 1000$ timesteps.

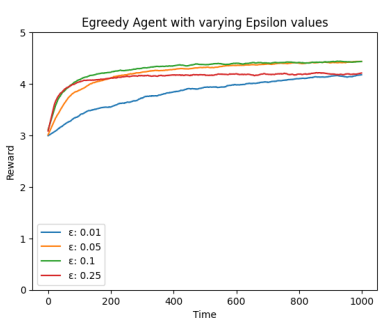

Figure 1: Learning curves of the ϵ-greedy agent for different ϵ values.

3.2 Optimistic Initialization

The Optimistic Initialization (OI) agent explores implicitly by setting initial Q-values higher than the expected reward. High initial values (e.g., 5 or 10) encourage the agent to select all arms at least once, as actual observed rewards are lower than the optimistic estimates. Lower initial values (e.g., 0 or 1) reduce this effect, causing the agent to exploit early and potentially miss high-reward arms.

Our experiments show that larger initial values lead to faster discovery of the best-performing arms, with cumulative rewards approaching the theoretical maximum more quickly. However, if the initial value is excessively high, the agent may spend more time exploring low-reward arms before converging, slightly delaying optimal performance. Thus, OI enables exploration without randomness but is sensitive to the initial Q-value.

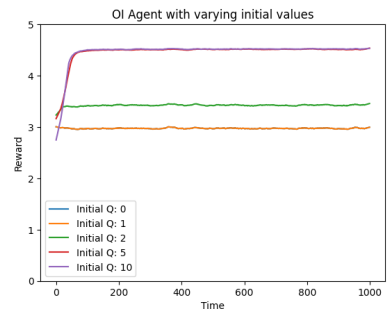

Figure 2: Learning curves of the Optimistic Initialization (OI) agent for different initial Q-values.

3.3 Upper Confidence Bound (UCB)

The UCB agent balances exploration and exploitation using an explicit uncertainty bonus. Actions with fewer selections receive a higher bonus, encouraging early exploration, while frequently selected arms rely more on the estimated Q-values. The parameter c controls the strength of this exploration term.

Small c values (e.g., 0.01) reduce the exploration bonus, causing the agent to behave more greedily, similar to exploitation-heavy ϵ-greedy agents. Large c values (e.g., 10) increase exploration, leading to more uniform action selection initially, which may delay reward accumulation. We observed that moderate c values (1.0–2.0) provided the best tradeoff, rapidly identifying high-reward arms while maintaining robust cumulative rewards over time.

3.4 Algorithm Comparison

Comparing the three agents, we find that:

- **Cumulative Reward:** UCB generally achieved the highest cumulative rewards, followed closely by Optimistic Initialization with carefully chosen initial values. ϵ-greedy performed well when ϵ was tuned properly, but it was more sensitive to suboptimal ϵ values.
- **Exploration Efficiency:** UCB provided the most efficient exploration, targeting uncertain actions intelligently. OI explored systematically due to high initial Q-values, whereas ϵ-greedy explored randomly, which can be less efficient.

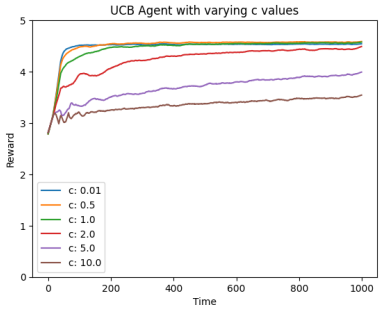

Figure 3: Learning curves of the UCB agent for different c values.

- **Parameter Sensitivity:** Both ϵ-greedy and OI required careful tuning of ϵ or initial Q-values to achieve strong performance. UCB was relatively robust across a moderate range of c values.

Overall, UCB's uncertainty-guided exploration offers the most consistent performance, while OI provides a simple, deterministic alternative that can achieve similar results if initial Q-values are chosen appropriately. ϵ-greedy is effective for small action spaces but may struggle in larger or more complex environments due to its reliance on random exploration.

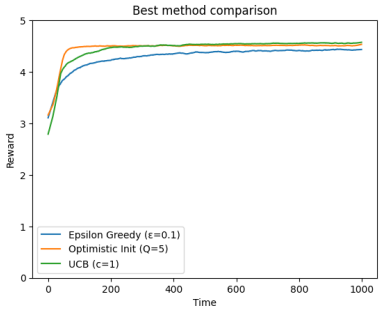

Figure 4: Learning curves for the best-performing version of each agent: ϵ-greedy (=0.1), Optimistic Initialization (Q0=5), and UCB (c=1). This plot shows cumulative performance over time.

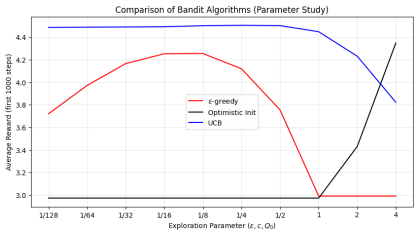

Figure 5: Comparison of average reward versus exploration parameter for each agent. The x-axis shows the exploration parameter (ϵ, Q_0, or c) on a log scale, while the y-axis shows the mean reward over the first 1000 timesteps. This highlights the sensitivity of each algorithm to its hyperparameter.

4 Discussion and Future Work

The three exploration strategies show tradeoffs in performance, stability, and parameter sensitivity:

- **ϵ-greedy:** Simple, relies on random exploration. Sensitive to ϵ: too small misses options, too large slows learning.
- **Optimistic Initialization (OI):** Deterministic, guided by high initial Q-values. Effective if initial values are moderate; excessive values delay convergence.
- **UCB:** Uncertainty-driven, robust, and consistently high-performing. Less sensitive to hyperparameters and stable across environments.

Potential directions to improve or extend the current study include:

- Non-stationary bandits: Track changing reward distributions over time.
- Thompson Sampling: Bayesian approach for uncertainty-guided exploration.
- Combining OI with ϵ-greedy: Mix structured and random exploration for faster convergence.
- Contextual bandits: Incorporate state information to make more informed decisions.

5 Conclusion

From our experiments and analysis, we can summarize the key findings:

- **What did we learn?** Efficient exploration is crucial for maximizing cumulative rewards. Random exploration works but is less efficient than structured or uncertainty-driven approaches.
- **Which algorithm works best?** UCB provides the most consistent and high-performing results. OI can achieve similar performance if initial values are tuned correctly, while ϵ-greedy is effective in simple settings.
- **When to use each method?**
  - Small action spaces: ϵ-greedy is sufficient.
  - Deterministic exploration needed: OI is simple and effective.
  - Large or uncertain environments: UCB is preferred for stability and overall performance.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction.* MIT press, 2018.

[2] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2-3, pp. 235–256, 2002.